

RedLightGreenLight EEG Projekt

Willkommen beim RedLightGreenLight EEG Projekt. Diese Anwendung nutzt EEG-Signale (Elektroencephalographie), um das Spiel "Red Light, Green Light" zu steuern. Hierbei wird das Verhältnis zwischen Beta- und Alpha-Wellen gemessen, um die Intention des Nutzers zu bestimmen. Ist der Nutzer entspannt, so ist das Verhältnis niedrig und die Figur bleibt stehen. Ist der Nutzer konzentriert, so ist das Verhältnis hoch und die Figur bewegt sich. Die beiden Zustände können durch das Schließen der Augen (entspannt) und das Öffnen der Augen (konzentriert) herbeigeführt werden. Alternativ können auch auf dem Bildschirm angezeigte Rechenaufgaben gelöst werden, um den konzentrierten Zustand herbeizuführen. Letzteres ist jedoch weniger zuverlässig.

Schnellstart

1. **Anforderungen installieren:** `bash pip install -r requirements.txt`
2. **Anwendung starten:** `bash python Main.py`

Dokumentation

Für ein detailliertes Verständnis der Projektstruktur und der Signalverarbeitung lesen Sie bitte die folgenden Dokumente:

- **ProjectStructure.pdf:** Erklärt die interne Architektur, die Signalverarbeitungspipeline und die BCI-Steuerung mit detaillierten **SVG-Diagrammen**.
- **GameDetails.pdf:** Details zur Spielimplementierung, MVC-Struktur und verwendeten Design-Patterns.

Funktionen

- **Echtzeit-EEG-Verarbeitung:** Übersetzt Gehirnaktivität durch spezialisierte Spektralanalyse in Spielbewegungen.

- **Interaktive Kalibrierung:** Personalisierte Schwellenwertbestimmung basierend auf entspannten und konzentrierten Zuständen.
- **Kanalverhältnis-Analyse:** Nutzt das Verhältnis zwischen Frontal-Beta (Kanal 1) und Okzipital-Alpha (Kanal 8) für eine hochzuverlässige Fokus-Erkennung.
- **Zeitbasierte Entprellung (Debounce):** Verhindert versehentliche Bewegungs-Trigger durch kontinuierliche Zustandsverifizierung.
- **Mock-Modus:** Integrierte Unterstützung für Tests ohne EEG-Hardware.

Fehlerbehebung & Feinabstimmung

Wenn sich die BCI-Steuerung träge oder instabil anfühlt, können Sie diese Kernparameter anpassen:

1. Empfindlichkeit anpassen

Wenn es zu schwer oder zu einfach ist, den "Konzentriert"-Zustand zu erreichen.

- **Datei:** Calibration/CalibrationModel.py
- **Parameter:** `self_sensitivity` (Standard: 0.7)
- **Aktion:**
 - **Zu schwer?** Verringern (z.B. 0.5 oder 0.6).
 - **Zu einfach?** Erhöhen (z.B. 0.8).

2. Stabilität (Debounce-Dauer)

Steuert, wie lange ein Zustand kontinuierlich gehalten werden muss, um eine Änderung auszulösen.

- **Datei:** EEG/RealTimeProcessor.py
- **Parameter:** `self_duration_threshold` (Standard: 0.5 Sekunden)
- **Aktion:**
 - **Flackernder Zustand?** Erhöhen auf 0.7 oder 1.0 .
 - **Verzögerte Reaktion?** Verringern auf 0.3 (Warnung: kann Fehlalarme erhöhen).

3. Hysterese-Marge

Die "Pufferzone" um den Schwellenwert, um schnelles Umschalten zu verhindern.

- **Datei:** Calibration/CalibrationModel.py
- **Parameter:** self_margin_ratio Berechnungsfaktor (Standard: 0.2)
- **Aktion:**
 - **Instabiles Schalten?** Faktor erhöhen (z.B. 0.3).

4. Frequenzbänder

Anpassen, wenn individuelle Unterschiede die Erkennung beeinflussen.

- **Datei:** EEG/SignalProcessor.py
- **Parameter:** self_alpha_band , self_beta_band
- **Aktion:** Bereiche verschieben (z.B. Alpha 7-13 , Beta 15-30), um Rauschen zu vermeiden.